



Materi Golang Pertemuan 9

Membahas web server Golang dengan net/http dan pengenalan Gin, termasuk handler, routing, request-response, JSON, dan latihan praktik.



Juara Coding

June 2026



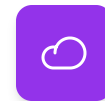
Pengenalan HTTP Native

Golang menyediakan package *net/http* untuk membangun server HTTP sederhana tanpa framework tambahan.

Komponen Dasar net/http

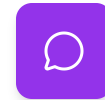


Komponen utama HTTP native mencakup handler, server, `ResponseWriter`, dan `Request` untuk menangani alur request-response.



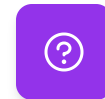
Handler dan Server

`http.HandleFunc` mengatur logika endpoint, sedangkan `http.ListenAndServe` menjalankan server HTTP.



ResponseWriter

`ResponseWriter` digunakan untuk mengirim output atau respons dari server ke client.



Request

`Request` menyimpan method, URL, query, header, dan body dari permintaan client.

```
import (
    "fmt"
    "net/http"
)

func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, World!")
}

func main() {
    http.HandleFunc("/", handler)
    fmt.Println("Starting server on :8080")
    http.ListenAndServe(":8080", nil)
}
```

Contoh HTTP Server Native

Server HTTP native sederhana di Golang ini membuat endpoint / pada port *8080* dan merespons dengan pesan hello.

Menangani Routing dan Method

```
fmt.Fprintf(w, "Welcome to the home page")
}

func getHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "GET request received")
}

func postHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "POST request received")
}

func putHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "PUT request received")
}

func deleteHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "DELETE request received")
}

func main() {
```

01 Routing dasar

Path didaftarkan ke handler dalam *net/http* untuk menangani permintaan sesuai URL yang masuk.

02 Pemeriksaan method manual

Perbedaan *GET* dan *POST* biasanya dicek langsung lewat *r.Method* di dalam handler.

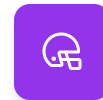
03 Tantangan skala aplikasi

Pendekatan ini fleksibel, tetapi pada aplikasi besar kodenya bisa makin panjang dan butuh struktur rapi.

Response JSON di HTTP Native

```
func jsonHandler(w http.ResponseWriter, r *http.Request) {  
    response := map[string]interface{}{  
        "message": "Hello, World!",  
        "status": 200,  
    };  
    w.Header().Set("Content-Type", "application/json")  
    w.WriteHeader(http.StatusOK)  
    json.NewEncoder(w).Encode(response)  
}  
  
func main() {  
    http.HandleFunc("/json", jsonHandler)  
    http.ListenAndServe(":8080", nil)  
}
```

Response JSON dikirim lewat header Content-Type dan ditulis ke ResponseWriter memakai encoding/json pada net/http.



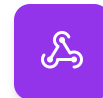
Set Header JSON

Tambahkan Content-Type agar client mengenali respons sebagai JSON.



Encode ke Writer

Pakai encoding/json untuk mengubah struct lalu kirim ke ResponseWriter.



Dipakai di REST API

Teknik ini umum untuk membangun REST API sederhana dengan net/http.

Keterbatasan Pendekatan Native



Routing Dinamis Terbatas

HTTP native baik untuk dasar server, tetapi routing dinamis membutuhkan penanganan kode tambahan.



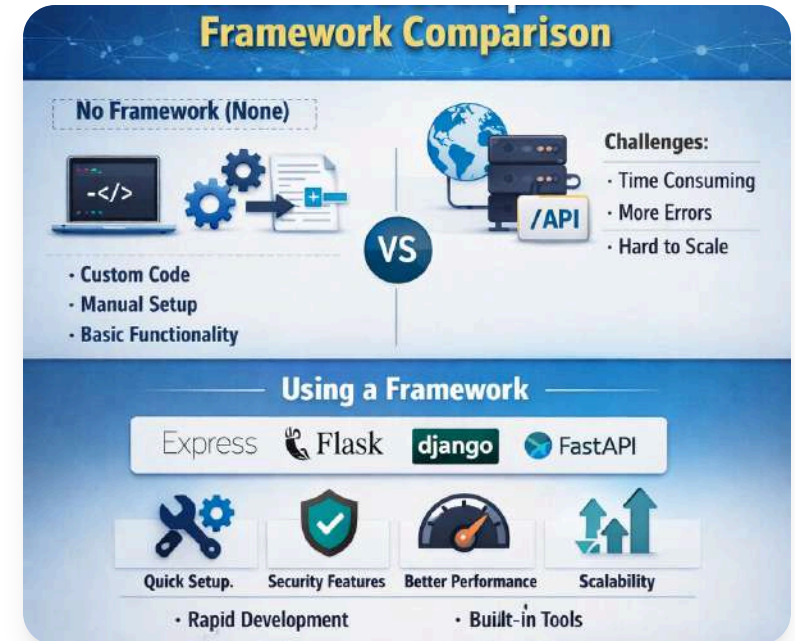
Framework Mempercepat Pengembangan

Framework seperti Gin membantu membuat aplikasi web dan API lebih cepat dan lebih terstruktur.



Fitur Umum Belum Siap Pakai

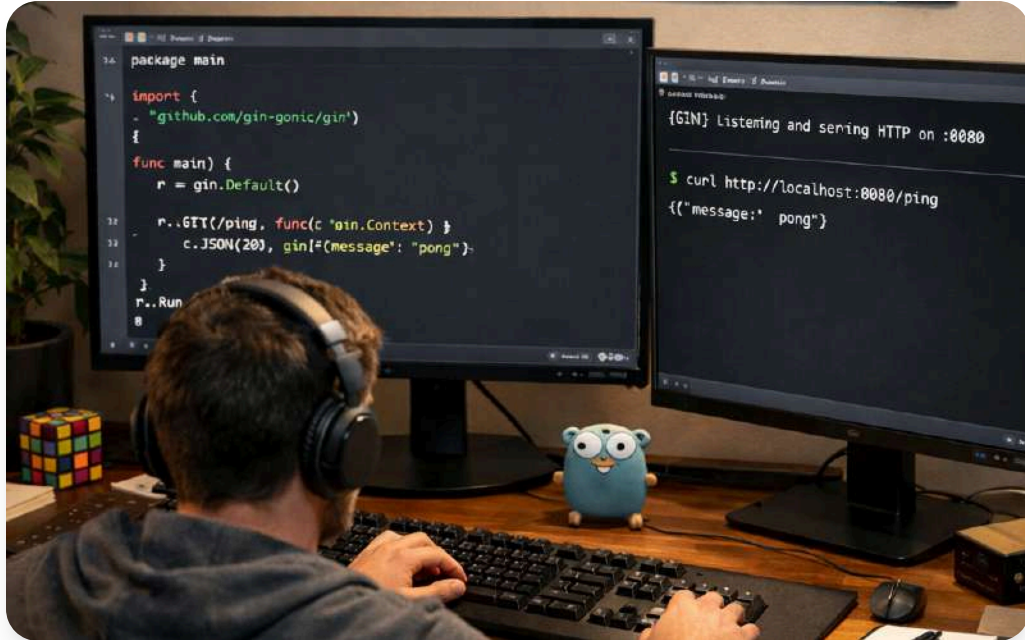
Middleware, validasi, dan grouping route perlu dibangun atau diatur sendiri oleh developer.





Pengenalan Gin Framework

Gin adalah framework web Golang yang ringan, cepat, dan populer untuk membangun REST API dengan routing dan middleware yang praktis.



Instalasi dan Setup Gin

Instal Gin dengan perintah `go get -u github.com/gin-gonic/gin`, lalu import package untuk mulai memakai engine, routing, dan helper response.

```
    "github.com/gin-gonic/gin"  
    "net/http"  
}  
  
func main() {  
    r := gin.Default()  
  
    r.GET("/ping", func(c *gin.Context) {  
        c.JSON(http.StatusOK,  
            gin.H{  
            }  
        })  
    }  
  
    r.GET("/hello/:name", func(c *ginContext) {  
        name := c.Param("name")  
        c.JSON(http.StatusOK, gin.H{  
            "message": "Hello " + name, "  
        })  
    }  
} r.Run(":8080")
```

Contoh Server Gin Sederhana

Server sederhana dengan Gin menyiapkan endpoint GET / yang mengembalikan respons JSON dan berjalan pada port 8080.

Keunggulan Gin



Routing Lebih Ringkas

Sintaks routing dan response di Gin lebih singkat dibanding penggunaan HTTP native.



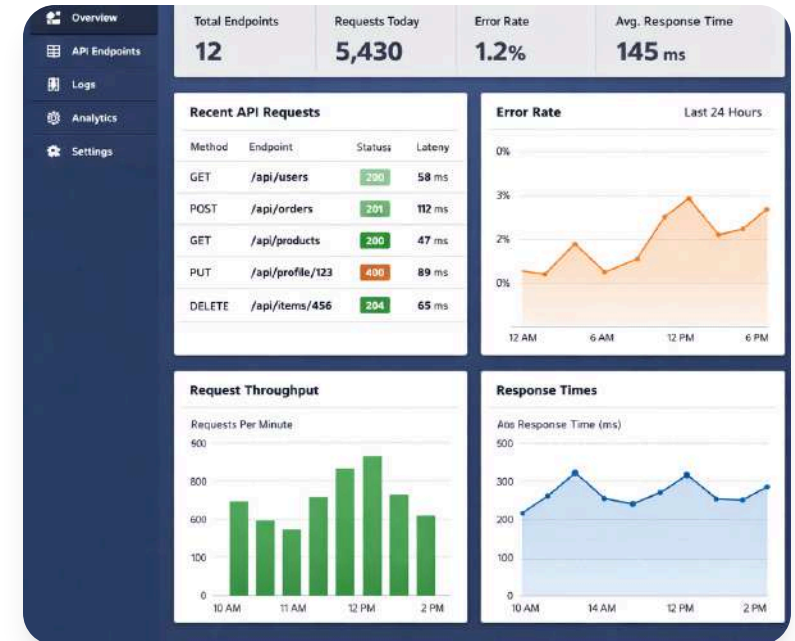
Produktivitas Lebih Tinggi

Cocok untuk pengembangan API yang cepat, terstruktur, dan efisien dalam implementasi.



Fitur Bawaan Lengkap

Gin mendukung middleware, parsing parameter, grouping endpoint, dan pengiriman JSON dengan mudah.



Perbandingan HTTP Native dan Gin

Aspek	HTTP Native	Gin
Tujuan utama	Belajar dasar HTTP	Pengembangan lebih cepat
Dependensi	Minimal	Framework tambahan
Routing	Sederhana	Lebih kompleks
Abstraksi	Lebih rendah	Lebih tinggi
Kemudahan penggunaan	Lebih manual	Lebih praktis

HTTP native ideal untuk layanan sederhana dan pembelajaran dasar, sedangkan Gin lebih sesuai untuk aplikasi dengan kebutuhan routing kompleks dan efisiensi pengembangan.

Latihan Praktik 1

```
func helloHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Hello, World!")
}

func dataHandler(w http.ResponseWriter, r *http.Request) {
    jsonData := {"message": "Here is some data!"}
    w.Header().Set("Content-Type", "application/json")
    fmt.Fprintln(w, jsonData)
}

func statusHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    fmt.Fprintln(w, "Status: OK")
}

func main() {
    http.HandleFunc("/hello", helloHandler)
```

01 Buat 2 endpoint

Gunakan *net/http* untuk membuat GET */* berisi teks sambutan dan GET */profil* berisi data JSON.

03 Uji hasil aplikasi

Cek respons melalui browser atau *Postman* untuk memastikan endpoint teks dan JSON berjalan benar.

02 Jalankan di port 8080

Konfigurasikan server agar berjalan pada port *8080* sehingga mudah diakses saat pengujian.

Latihan Praktik 2

01 Endpoint utama

GET `/` menampilkan pesan utama sebagai respons sederhana dari aplikasi Gin.

03 Data user JSON

GET `/user` mengembalikan data user dalam format JSON, server berjalan di port 8080 lalu dibandingkan dengan HTTP native.

02 Sapaan dinamis

GET `/hello/:nama` menampilkan sapaan berdasarkan parameter nama yang dikirim pada URL.

Endpoints:	JSON Responses:
GET /ping	Response for /ping:
url http://localhost:8080/ping	<pre>{ "message": "pong"}</pre>
GET /user/123	Response for /user/123:
url http://localhost:8080/user/123	<pre>{ "id": 123, "name": "John Doe", "email": "john@example.com"}</pre>
POST /login	Response for /login:
url -X POST http://localhost:8080/login \ -d '{"username": "admin", "password": "1234"}\' \ -H "Content-Type: application/json"	<pre>{ "status": "success"}</pre>

Penutup



"net/http memberi fondasi, Gin memberi kecepatan dan struktur."

Kesimpulan Sesi

